UI extensibility

Create a settings page

Learn how to create a settings page for your app.

You can create a React-based settings page for your app on the latest version (`2025.2`) of the developer platform, that users who install your app can navigate to and customize for their account.

This guide walks you through how to build a settings component, using React, which replaces the **previous settings experience**. If you deploy your new settings component, users who install your app going forward will immediately see your new settings extension instead of any previously built settings page.

Before you begin, follow the **quickstart guide** to create your first app on the latest version of the developer platform.

> **Please note:** if you previously created a **settings page** for your legacy app, you'll need to refactor your settings experience using the new configuration options outlined in this guide. It's recommended that you note all of the interface elements in the current production version of your app before you start configuring your new settings component on the latest version of the developer platform. Once you move to the new settings component for your app, you'll lose access to the previous WYSIWYG configuration UI you previously used.

Prerequisites

Ensure you've installed the latest beta version of the **HubSpot CLI**.

If you haven't done so yet, **create a new app**.

Create an app settings page

1. Add an app settings component to your project

HubSpot Developers

Then, when prompted to select a component to add, select **Settings**.

```
? [--type] Select a component to add:  (Press <space> to select, <a> to toggle all,
 <i> to invert selection, and <enter> to proceed)
 ○ Card
 ○ App Function
❯○ Settings
 ○ Webhook
 ○ Custom Workflow Action
```

A new `settings/` directory will be created in the project's `src/app/` directory. The `settings/` directory will contain:

- A JSON configuration file (`*-hsmeta.json`)
- A React component file (`.jsx`)
- A `package.json` file

```
myProject
└── src/
    └── app/
        └── settings/
            ├── new-settings-page-hsmeta.json
            ├── NewSettingsPage.tsx
            └── package.json
```

-hsmeta.json NewSettingsPage.tsx package.json

HubSpot Developers

```
    "config": {
      "entrypoint": "/app/settings/NewSettingsPage.tsx"
    }
  }
```

2 Upload to HubSpot

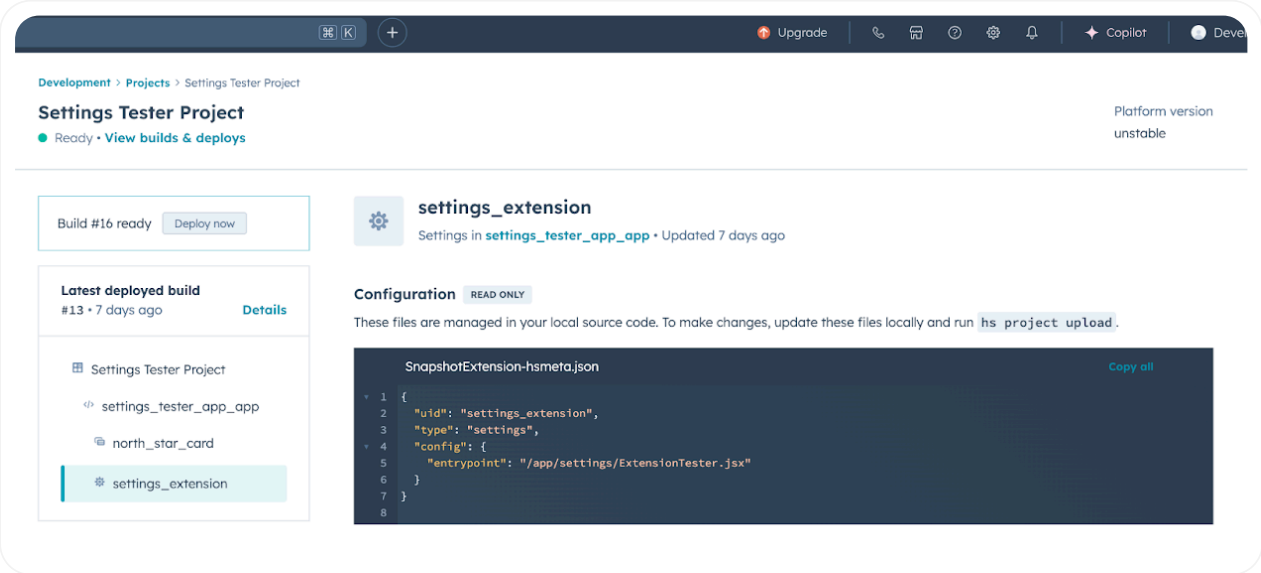
To upload your settings page to HubSpot:

Run `hs project install-deps` from within your local project directory to install necessary dependencies. This will create a `package-lock.json` file, which will speed up the build of the uploaded settings extension, as well as ensure that any dependencies in your local development environment and production match.

Then, run `hs project upload`.

After the project finishes deploying, open the project in HubSpot by running `hs project open`. Alternatively, in HubSpot you can navigate to **Development** > **Projects**, then click the **name** of your project.

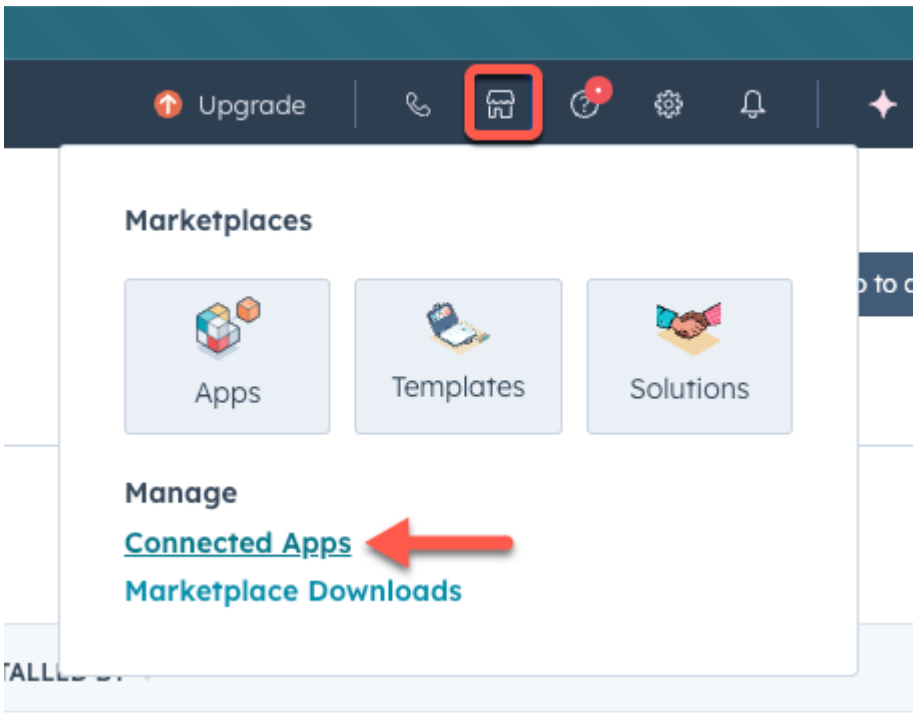
Your settings component should now be listed on the details page.



HubSpot Developers

›

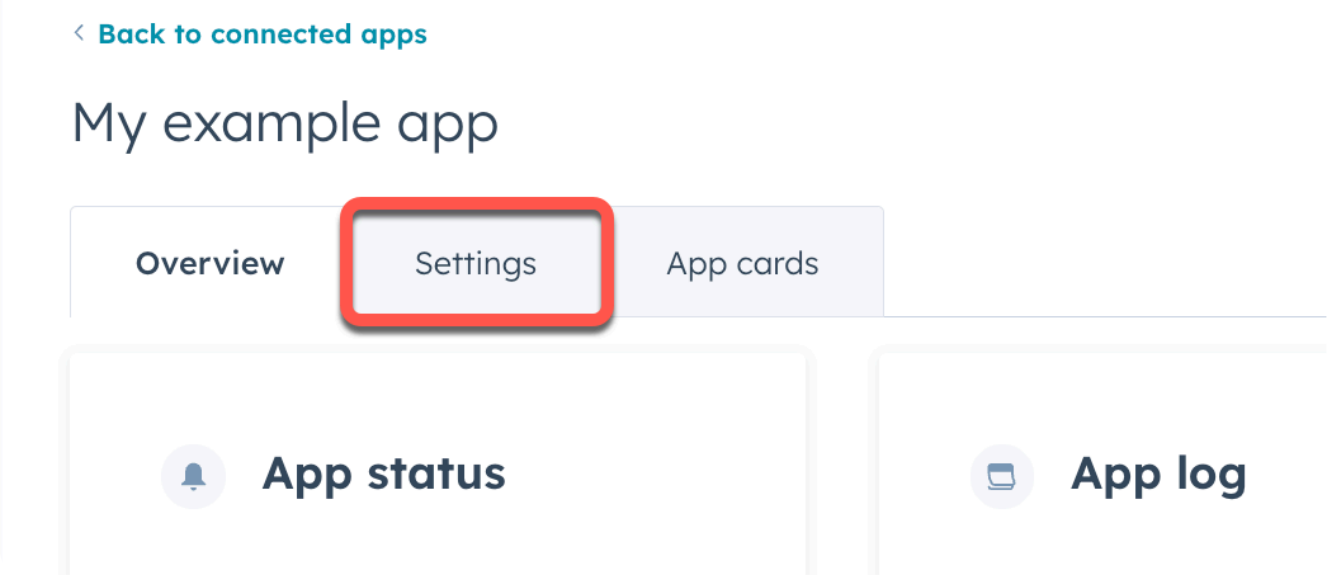
then click **Connected apps**.



Click the **My apps** tab to view a list of the account's currently installed apps.

Click the **name** of your app, which will redirect you to your app's overview page.

On the overview page, click the **Settings** tab.



Use `hubspot.fetch` to leverage your backend to save and retrieve settings. Learn more about using this approach in the legacy documentation.

Check out the reference documentation on **standard components** for how to use React components when building your extension, or use the component in the **Figma Design Kit**.

Use the `hs project dev` command to iteratively build out your settings page and preview your changes locally.

> ⚠ The local development server will only pick up changes saved to the front-end React file. If you update the `*-hsmeta.json` or `package.json` files, you'll need to stop the server, upload your changes, then start the server again.

Component best practices

The sections below outline several best practices to keep in mind as you build out the settings experience for your app.

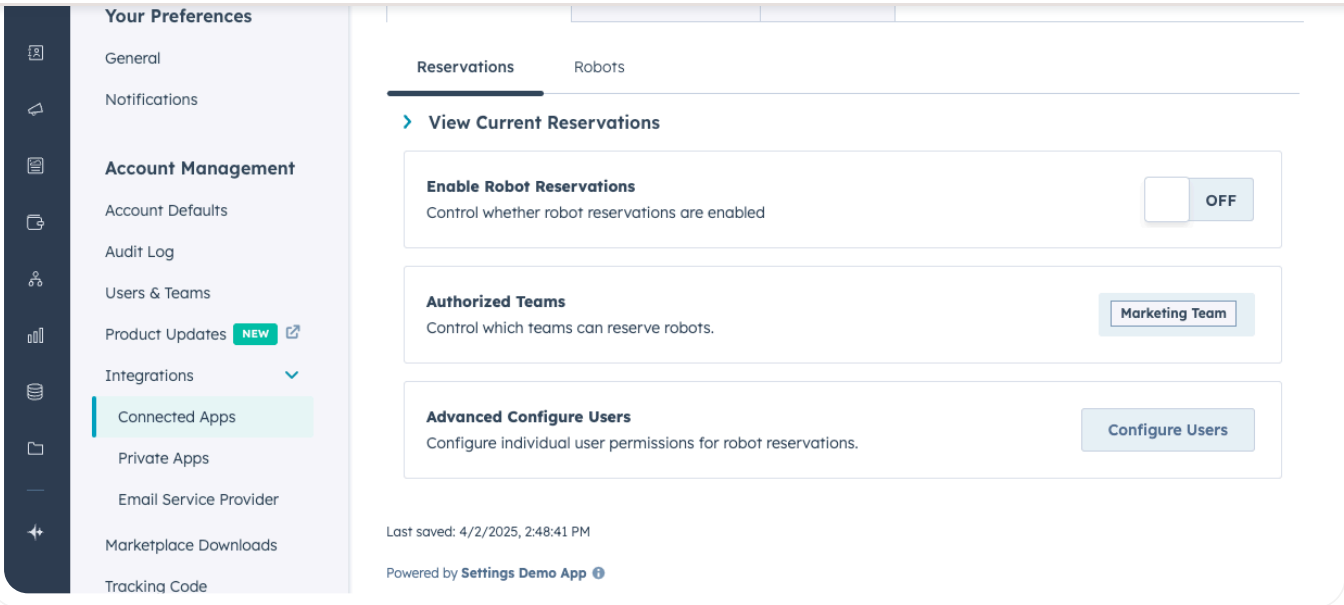
Organizing content

If you have enough content in your settings extension to warrant the need to separate and organize all of the user's settings data, you should consider using the **Panel**, **Modal**, **Accordion**, and **Tabs** components accordingly. Consider how you want to present and arrange your settings and the corresponding data that should be fetched for each component.

Tabs

If you're using **Tabs**, use the `default` tab variant. The settings extension is already contained within an enclosed variant tab, and a second layer of enclosed tabs will visually clash with the design.

HubSpot Developers



The following snippet outlines how to structure your tabs.

```
<Tabs>
  <Tab title="First Tab">
    <Text>Here is the content of the first tab.</Text>
  </Tab>
  <Tab title="Second Tab">
    <Text>This is where the content of the second tab goes.</Text>
  </Tab>
</Tabs>;
```

Refer to for more best practices, guidelines, and code examples to get you started. HubSpot also provides design pattern guidance for , , and component usage.

Last modified on January 28, 2026

HubSpot Developers

Create an app home page
Previous

Logging and monitoring for UI extensions
Next

HubSpot Developers
Copyright © 2026 HubSpot, Inc
Privacy Policy
Security
Website Accessibility
Manage Cookies